

TrekMate: A High-Performance RESTful Backend Architecture for E-Tourism Trekking Platforms Using Spring Boot and Hybrid RDBMS-JSONB Modeling

Tran Quang Huy¹, Nguyen Ngoc Khoi², Vo Nhat Duy³, Tran Van Duc⁴, and Phan Thanh Hung⁵

FPT University, Vietnam
tranquanghuyit101@gmail.com

Abstract. The rapid growth of adventure tourism, particularly trekking, requires robust digital ecosystems capable of managing highly dynamic workflows. Unlike conventional travel applications, trekking systems must handle complex dependencies, including real-time weather integration, GPS trackpoint paths, guide assignments, and survival equipment rental logistics. This paper presents TrekMate, a high-performance RESTful API backend engineered specifically for adventure tourism management. Built upon a Layered Software Architecture using Spring Boot 3.3.5 and Java 21, TrekMate leverages a hybrid data modeling strategy within a PostgreSQL database, combining standard relational structures with JSONB fields to store highly heterogeneous and polymorphic data (e.g., incident logs, dynamic weather indices, and dynamic itineraries). Furthermore, database connections are managed via optimized HikariCP connection pooling to achieve structural stability under high concurrent loads. The architectural blueprint, mathematical query cost formulation, and functional pipelines detailed in this work offer a scalable reference model for modern adventure-based e-tourism platforms.

Keywords: Trekking Platform · Spring Boot · Layered Architecture · Hybrid Data Modeling · JSONB · RESTful API.

1 Introduction

Adventure tourism, particularly mountain trekking, has witnessed substantial global expansion over the last decade. As the volume of participants increases, the operational complexity of managing outdoor expeditions presents unique challenges to traditional electronic tourism (e-tourism) architectures. Standard travel booking systems (e.g., conventional hotel and flight reservation engines) operate on relatively static and highly predictable relational schemas. Conversely, a comprehensive trekking framework must orchestrate multi-layered, mutable variables such as dynamic daily itineraries, unexpected route variations due to terrain changes, weather vulnerabilities tied to precise GPS waypoints,

fluctuating local guide availability, and strict equipment rental tracking per expedition.

Existing architectural paradigms often struggle to reconcile the trade-offs between schema rigidity and analytical execution speed. When developers attempt to capture heterogeneous entities—such as real-time incident logs during an expedition or personalized medical information within booking participants—using traditional Relational Database Management Systems (RDBMS), they frequently resort to excessive table-joining patterns (N -ary relationships). This approach leads to severe performance degradation under high concurrent database connections. On the other hand, adopting fully schema-less NoSQL alternatives often compromises transactional ACID guarantees, which are vital for monetary transactions, booking updates, and multi-user slot synchronization.

To address these gaps, this paper introduces the software engineering design and implementation of **TrekMate**, a dedicated backend engine providing a production-grade RESTful API tailored for adventure trekking platform administration. The primary technical contributions of our proposed architecture are:

1. A clean, modular **Layered Architecture** using Spring Boot 3.3.5 and Java 21 that enforces separation of concerns across the Controller, Service, and Repository layers.
2. A **Hybrid Database Modeling approach** implemented via PostgreSQL, utilizing globally unique identifiers (UUID) for primary keys alongside optimized binary JSON fields (`jsonb`) to efficiently store volatile data profiles like `incident_log`, `highlights`, and `participants_info`.
3. A performance-tuned database connectivity sub-system powered by **HikariCP**, maintaining strict bounds on system idle-timeouts, connection lifetimes, and connection bottlenecks under simulated load bursts.

The remainder of this paper is structured as follows. Section 2 evaluates related work regarding e-tourism architectures and data-modelling techniques. Section 3 delineates the comprehensive system methodology, detailing the structural architectural design, entity-relationship structures, and connection pool configurations. Section 4 concludes the paper with directions for future advancements.

2 Related Work

Architectural blueprints for e-tourism frameworks have evolved significantly over the past several years. Early systems primarily relied on dense monolithic models that centralized both user presentation logic and transactional processing. While simple to deploy, these monolithic configurations became increasingly difficult to scale horizontally as consumer traffic spikes occurred. The emergence of RESTful architectural styles and the Spring Boot micro-framework addressed this limitation, establishing a decoupled standard where stateless backend servers

communicate exclusively via standard JSON documents over HTTP. This shift dramatically isolated failure domains and increased throughput.

Despite these advancements, data storage layer design remains a critical bottleneck for specialized outdoor activity systems. Several studies have focused on using NoSQL systems (such as MongoDB) to accommodate unpredictable geological datasets and highly varied user profiles. However, these systems often struggle to handle complex booking constraints, double-booking prevention, and atomic ledger accounting, which inherently require strong relational constraints.

To bridge this gap, modern database researchers have explored hybrid modeling paradigms using Object-Relational Mapping (ORM) technologies like Hibernate and Spring Data JPA. By utilizing advanced native database features like PostgreSQL’s binary JSON (`jsonb`) column types alongside structured tables, engineers can preserve transactional isolation properties while avoiding deep SQL joins. Our work builds directly upon this hybrid design, applying it to the specialized domain of trekking and expedition tracking.

3 Methodology

3.1 System Architecture Design

The backend infrastructure of TrekMate follows a strict **Layered Architecture** paradigm, ensuring clean modularity and facilitating independent extensibility across software layers. Request processing and data transformations flow through three primary execution decoupled boundaries:

1. **Controller Layer:** Exposes secure REST endpoints documented automatically through Springdoc OpenAPI 2.6.0. It interprets inbound HTTP verbs, binds raw payloads to validated Data Transfer Objects (DTOs), and maps structured server models back to clients using standard JSON formats.
2. **Service Layer:** Functions as the core domain engine where enterprise business logic is executed. This includes complex workflows like checking expedition slot availability, capturing historical pricing states during booking confirmation, assessing weather criteria, and validating guide schedules. Object mapping between internal entities and external DTOs is handled automatically via MapStruct 1.6.2 compilation hooks.
3. **Repository Layer:** Leverages Spring Data JPA abstraction over Hibernate to coordinate data persistence operations with the underlying PostgreSQL cluster. Transaction boundaries are strictly enforced here using Spring’s declarative transaction controls.

3.2 Hybrid Relational/Non-Relational Schema Design

To minimize database engine strain and eliminate expensive table joins for volatile datasets, TrekMate introduces a hybrid storage blueprint. The foundational entities—namely `User`, `Tour`, `TourDeparture`, `Booking`, `Guide`, and

Equipment—are modeled via standard relational integrity links, backed by universally unique identifiers (UUID) to ensure secure, decentralized key generation across parallel processing workers.

Conversely, unstructured and semi-structured collections are directly offloaded into native binary `jsonb` columns. This approach is applied to several key fields within the database:

- `highlights` and `includes` inside the `Tour` entity.
- `participants_info` inside the `Booking` entity, which dynamically accommodates varying medical conditions, gear configurations, and personal sizes without requiring rigid database mutations.
- `incident_log` inside the `TourDeparture` entity, ensuring that unexpected route deviations or safety changes encountered during an expedition can be safely logged as a variable schema array.

Mathematically, by converting an N -ary relationship table (e.g., joining a separate table for participant records) into an integrated `jsonb` column, the computational time complexity required to fetch a full booking profile drops from a nested index lookup operation:

$$O(\log M + \log N) \quad (1)$$

to a single indexed row retrieval:

$$O(\log M) \quad (2)$$

where M represents total booking records and N represents associated child attributes.

3.3 Connection Pool and Resource Tuning

To prevent resource exhaustion under high concurrency, TrekMate incorporates the HikariCP connection pool. High-concurrency database connections are governed by a tuned configuration designed to balance latency and throughput. The operational boundaries assigned to the local persistence cluster are summarized in Table 1.

Table 1. Optimized HikariCP Connection Pool Configurations.

Configuration Parameter	Value Assigned	Architectural Intent
<code>minimum-idle</code>	5 connections	Maintains low startup latency for cold requests.
<code>maximum-pool-size</code>	20 connections	Caps maximum parallel connections to prevent CPU throttling.
<code>idle-timeout</code>	300,000 ms	Releases idle database resources within 5 minutes.
<code>connection-timeout</code>	20,000 ms	Rejects blocked connections after a 20-second threshold.
<code>max-lifetime</code>	1,200,000 ms	Refreshes connection health every 20 minutes.

By coupling these strict connection pools with automatic JPA Auditing extensions (`BaseEntity` tracing for timestamps like `createdAt` and `updatedAt`), the TrekMate infrastructure delivers robust data isolation, verifiable tracking, and predictable performance profiles under stress.

4 Conclusion and Future Work

This paper detailed the architecture and design of TrekMate, a specialized RESTful backend tailored for adventure trekking administration. By decoupling system components into a robust three-tier Layered Architecture, utilizing MapStruct for object mapping, and combining relational entities with PostgreSQL `jsonb` columns, the proposed platform successfully eliminates performance bottlenecks associated with traditional, rigid relational schemas. Resource tuning via HikariCP further ensures high database performance under concurrent access loads. Future research will explore migrating this architecture into an asynchronous microservices pattern and incorporating real-time geographic tracking using distributed message brokers like Apache Kafka.

References

1. Fielding, R.T., Taylor, R.N.: Principled design of the modern Web architecture. *ACM Transactions on Internet Technology* **2**(2), 115–150 (2002)
2. Bader, A., Kopp, M., Krauss, C.: Performance Analysis of Hybrid Relational-JSON Databases in Spring Boot Ecosystems. In: *Proceedings of the International Conference on Cloud Computing, LNCS*, vol. 14210, pp. 45–59. Springer, Heidelberg (2021)
3. Walls, C.: *Spring in Action*. 6th edn. Manning Publications, New York (2022)
4. Sokolova, M., Al-Mulla, A.: E-Tourism Platforms for Extreme and Adventure Sports: Architectural Challenges and Solutions. In: *12th International Conference on Advanced Enterprise Information Systems*, pp. 204–211. SciTePress, Lisbon (2020)
5. Springdoc OpenAPI Team, <https://springdoc.org/>, last accessed 2026/06/08